

Busca Adversarial - Minimax

Nota de Aula 04

Prof. Eduardo Bezerra

Sumário

1	Do Agente Único ao Agente Adversarial	3
1.1	O problema da existência de oponentes	4
2	Tipos de Jogos	4
2.1	Escopo destas notas	5
3	Formulação Formal	6
4	Árvore de Jogo	7
4.1	Estrutura	7
4.2	Diferença em relação à árvore de busca monoagente	8
5	Valor Minimax	9
5.1	Exemplo resolvido: calculando valores minimax	10
6	Algoritmo Minimax	11
6.1	Implementação	11
6.2	Exemplo resolvido: execução recursiva do Minimax	12
6.3	Propriedades	14
7	Poda Alfa-Beta	15
7.1	Ideia central	15
7.2	Por que a poda não altera o resultado	16
7.3	Exemplo resolvido: poda alfa-beta passo a passo	16
7.4	Algoritmo	18
7.5	Eficiência e ordem de varredura	19
8	Funções de Avaliação Heurística	20
8.1	A ideia	20
8.2	Requisitos de uma boa função de avaliação	21
8.3	Funções de avaliação lineares ponderadas	21
8.4	Exemplo 1	22
8.5	Exemplo 2	24
8.6	Funções de avaliação não lineares	25
8.7	Aprendizado de funções de avaliação	25
9	A Importância da Profundidade	25

10 Busca Quiescente	25
10.1 O efeito de horizonte	26
10.2 A solução: busca quiescente	26
11 Resumo	28
12 Atividades Intra-Classe	28
12.1 Aquecimento	28
12.2 Cálculo do valor Minimax e poda alfa-beta	29
12.3 Exercício de poda alfa-beta	29
12.4 Projeto de função de avaliação	30

Objetivos

Ao final da leitura, você deve ser capaz de:

Motivação e formulação

- Explicar por que os algoritmos de busca de agente único não se aplicam diretamente a ambientes com oponentes.
- Formular um jogo determinístico de dois jogadores como um problema de busca adversarial.

Valor minimax e política ★

- Definir o conceito de valor minimax de um estado e calcular esse valor recursivamente em uma árvore de jogo.
- Extrair a decisão ótima (política) a partir dos valores minimax calculados.

Algoritmo e complexidade ★

- Implementar o algoritmo Minimax e analisar suas propriedades de otimalidade e complexidade.
- Descrever o mecanismo de poda alfa-beta, executar o algoritmo passo a passo sobre uma árvore concreta e explicar por que ela não altera o resultado do Minimax.

Avaliação heurística e horizonte

- Justificar a necessidade de funções de avaliação heurística e descrever os requisitos de uma boa função de avaliação.
- Calcular o valor de uma função de avaliação linear para um estado concreto.
- Explicar o efeito de horizonte e a busca quiescente como resposta a esse problema.

Roteiro de leitura

Nesta leitura, você verá primeiro por que planos falham em jogos adversariais e o que os substitui (Seções 1–3). Em seguida, como modelar o problema formalmente e construir a árvore de jogo (Seções 3–4). Depois, o algoritmo Minimax e sua otimização via poda alfa-beta (Seções 5–7). Por fim, por que a busca exata é inviável em jogos reais e como funções de avaliação e busca quiescente contornam esse problema (Seções 8–10).

Os objetivos marcados com ★ são os mais centrais para a aula: concentre-se neles se o tempo for limitado.

1 Do Agente Único ao Agente Adversarial

Nos problemas de busca determinística estudados até aqui, o agente opera em um ambiente que podemos chamar de *monoagente*: há um único tomador de decisões, e o próximo estado depende apenas da ação executada pelo agente e das regras do ambiente, de forma previsível. Nesse contexto, o agente pode construir um *plano* (que corresponde a uma sequência de ações que leva do estado inicial ao objetivo) antes de agir e depois executá-lo sem surpresas.

Duas propriedades presentes nos problemas estudados até aqui tornavam esse modelo tratável:

- O agente possui *controle total* sobre suas ações e seus efeitos: toda ação executada produz exatamente o resultado esperado.
- O ambiente é *estático* durante o planejamento: nada muda enquanto o agente delibera.

Além disso, nesse modelo é frequentemente possível encontrar a solução ótima (o plano de menor custo) por meio de algoritmos como UCS ou A*.

1.1 O problema da existência de oponentes

Quando há um *oponente* no ambiente, o agente deixa de controlar todos os eventos relevantes para o resultado futuro. O ambiente pode continuar estático enquanto o agente delibera, no sentido de que nada muda durante o planejamento; a diferença essencial é que, entre duas decisões do agente, haverá decisões de outro jogador. MAX controla apenas suas próprias ações, e o resultado futuro depende também das escolhas de MIN.

A consequência imediata é que um *plano fixo, definido antecipadamente e sem reavaliação, não é suficiente* em um ambiente adversarial. Um plano é uma sequência de ações para o agente; mas entre as ações do agente haverá ações do oponente, e não há como prever com certeza quais serão. Se o agente simplesmente ignorar o oponente e executar um plano fixo, o oponente pode facilmente arruiná-lo.

O que é necessário, portanto, não é um plano, mas uma *política (policy)*: uma função que, dado um estado em que é a vez do agente jogar, indica qual é a melhor ação a tomar. A política cobre os estados de decisão do agente, incluindo aqueles que resultam de diferentes escolhas do oponente. Em outras palavras, em vez de decidir apenas “o que fazer agora e depois”, o agente precisa decidir “o que fazer em cada situação possível em que precise agir”.

Pergunta 1

Em um problema de busca monoagente, o agente constrói um plano e o executa. Por que essa abordagem falha quando há um oponente? Construa um exemplo no jogo da velha: defina uma sequência de 3 movimentos planejados por MAX (X) e mostre uma jogada de MIN (O) que torna esse plano inválido antes de ser concluído. Não basta descrever em palavras; desenhe os tabuleiros correspondentes.

Além da questão da política, a presença de um oponente introduz outra dificuldade: em jogos reais, há frequentemente um *limite de tempo* para cada decisão. O agente não pode deliberar indefinidamente; ele precisa tomar uma decisão dentro de um prazo, mesmo que essa decisão não seja ótima. Essas restrições tornam-se especialmente importantes em jogos, nos quais normalmente há um limite explícito de tempo por jogada.

2 Tipos de Jogos

O termo “jogo” é usado aqui no sentido da *Teoria dos Jogos*: qualquer situação de interação estratégica entre dois ou mais agentes com objetivos potencialmente conflitantes. Nesse sentido amplo, xadrez, damas, Go, pôquer e até negociações comerciais são “jogos”. Os jogos têm sido objeto de estudo em IA desde os seus primórdios, por razões históricas e técnicas. Do ponto de vista técnico, jogos de tabuleiro oferecem um ambiente formal, limpo e bem definido, ideal para o desenvolvimento e avaliação de algoritmos de tomada de decisão. Além disso, os jogos capturam o elemento essencial do ambiente adversarial: a presença de um oponente racional

cujas ações não são controladas pelo agente e podem ser escolhidas para prejudicá-lo. Histórica e culturalmente, jogar xadrez bem era tido como sinal incontestável de capacidade intelectual, o que motivou décadas de pesquisa em IA nesse domínio.

Jogos podem ser classificados ao longo de várias dimensões. A tabela abaixo resume essas dimensões e indica o impacto algorítmico de cada uma, deixando claro *por que* estas notas se restringem a um subconjunto específico de jogos:

Dimensão	Escopo desta nota	Caso mais geral	O que muda ao sair do escopo
Determinismo	Determinístico	Estocástico	Precisa modelar probabilidades; o valor de um estado passa a ser uma esperança, não um máximo/mínimo.
Jogadores	Dois	Múltiplos	Com mais de dois jogadores, já não basta alternar entre “maximizar” e “minimizar”; em geral, cada jogador passa a ter sua própria utilidade.
Soma	Soma zero	Soma geral	Com soma geral, os jogadores podem cooperar ou ter objetivos independentes; precisa de funções de utilidade separadas.
Informação	Perfeita	Imperfeita	O agente não observa diretamente o estado completo; precisa raciocinar sobre conjuntos de estados possíveis e, em modelos probabilísticos, sobre crenças acerca desses estados.

2.1 Escopo destas notas

Nestas notas, estudaremos busca adversarial no contexto de jogos: determinísticos, dois jogadores (denotados por MAX e MIN), soma zero, informação perfeita e turnos alternados (MAX faz o primeiro movimento, depois MIN, e assim por diante até o fim do jogo). Xadrez, damas e jogo da velha são exemplos clássicos que se enquadram nessa categoria.

Esse recorte é pedagógico, não apenas técnico: ele permite introduzir os conceitos centrais de busca adversarial sem sobrecarregar a modelagem com incerteza, informação incompleta ou múltiplos objetivos.

Por que soma zero simplifica o problema? Em um jogo de soma zero com dois jogadores, não é necessário definir duas funções de utilidade separadas. Basta definir uma única função $U(s)$ que representa a utilidade do estado s para MAX. A utilidade de MIN no mesmo estado é

simplesmente $-U(s)$. Isso significa que maximizar a utilidade de MAX equivale a minimizar a utilidade de MIN, e vice-versa; daí o nome do algoritmo que estudaremos, chamado *Minimax*.

Pergunta 2

O Minimax estudado nestas notas pressupõe dois jogadores com objetivos opostos. O que deixa de funcionar diretamente quando passamos para um jogo com três ou mais jogadores?

Explique por que já não é suficiente propagar um único valor alternando operações de máximo e mínimo. Que informação sobre os resultados dos diferentes jogadores precisaria ser representada? Discuta também se o conceito de jogo de soma zero ainda pode ser definido nesse caso.

3 Formulação Formal

Um jogo determinístico de dois jogadores pode ser formalizado como uma tupla:

$$\langle S, s_0, P, \text{PLAYER}, \text{ACTIONS}, \text{RESULT}, \text{TERMINAL-TEST}, \text{UTILITY} \rangle$$

onde:

- S é o conjunto de **estados** do jogo.
- $s_0 \in S$ é o **estado inicial**.
- $P = \{\text{MAX}, \text{MIN}\}$ é o conjunto de **jogadores**.
- $\text{PLAYER}(s)$ indica qual jogador deve agir em cada estado não terminal s .
- $\text{ACTIONS}(s)$ retorna o conjunto de **jogadas legais** disponíveis no estado s .
- $\text{RESULT}(s, a)$ é a **função de transição**: dado o estado s e a ação $a \in \text{ACTIONS}(s)$, retorna o estado resultante.
- $\text{TERMINAL-TEST}(s)$ é o **teste de estado terminal**: retorna verdadeiro se s é um estado em que o jogo terminou (vitória, derrota ou empate), e falso caso contrário.
- $\text{UTILITY}(s, p)$ é a **função utilidade**: para um estado terminal s e um jogador p , retorna um valor numérico que representa o resultado do jogo para p . No jogo da velha, por exemplo, os valores típicos são +1 (vitória), 0 (empate) e -1 (derrota).

Notação simplificada. Como estudaremos jogos de soma zero do ponto de vista de MAX, daqui em diante escreveremos simplesmente $\text{UTILITY}(s)$, entendendo que a utilidade é dada da perspectiva de MAX.

A *solução* para MAX em um jogo formalizado dessa forma é uma *estratégia* (ou *política*): uma função que associa a cada estado não terminal em que $\text{PLAYER}(s) = \text{MAX}$ uma ação legal naquele estado, isto é, $\pi(s) \in \text{ACTIONS}(s)$ para cada estado de decisão de MAX.

A tabela abaixo compara os componentes de um jogo com os de um problema de busca monoagente, para facilitar a próxima pergunta:

Componente	Busca monoagente	Jogo adversarial
Estados	S	S (o mesmo)
Ações	$ACTIONS(s)$	$ACTIONS(s)$ (o mesmo)
Transição	$T(s, a)$	$RESULT(s, a)$ (o mesmo)
Custo	$c(s, a)$	— (substituído por utilidade)
Terminal	$GoalTest(s)$	$TERMINAL-TEST(s)$
Objetivo	minimizar custo	$UTILITY(s)$ (nova)
Decisão	quem age	$PLAYER(s)$ (novo)
Solução	plano = sequência de ações	política $\pi(s)$ para os estados de decisão do agente (nova)

Pergunta 3

Com base na tabela acima, identifique cada diferença entre as duas formulações e explique a razão didática: o que o componente novo ou modificado captura que a formulação monoagente não conseguia?

Pergunta 4

Considere o jogo da velha. Identifique, nesse jogo, o que correspondem a $ACTIONS$, $RESULT$, $TERMINAL-TEST$, $UTILITY$ e $PLAYER$. Não é necessário escrever formalmente toda a tupla.

4 Árvore de Jogo

A estrutura central da busca adversarial é a *árvore de jogo* (*game tree*). Ela é análoga à *árvore de busca* (*search tree*) utilizada em problemas de agente único, mas com uma diferença fundamental: ao percorrer qualquer caminho da raiz até uma folha, os nós representam estados em que jogadores diferentes tomam decisões de forma alternada, refletindo a natureza sequencial e competitiva do jogo.

A Figura 1 ilustra a estrutura de uma árvore de jogo, destacando a alternância de decisões entre os jogadores ao longo de um caminho da raiz até uma folha. Cada nível da árvore corresponde ao turno de um jogador específico: nós rotulados como MAX representam estados em que o agente busca maximizar a utilidade, enquanto nós rotulados como MIN representam estados em que o adversário busca minimizá-la. Essa alternância sequencial é fundamental para o funcionamento de algoritmos como o minimax.

4.1 Estrutura

Uma árvore de jogo possui as seguintes características:

- Cada nó representa um estado do jogo.
- A raiz representa o estado inicial s_0 .
- Cada aresta representa uma jogada (ação) de um dos jogadores. Cada caminho da raiz até uma folha representa uma sequência alternada de jogadas de MAX e MIN.

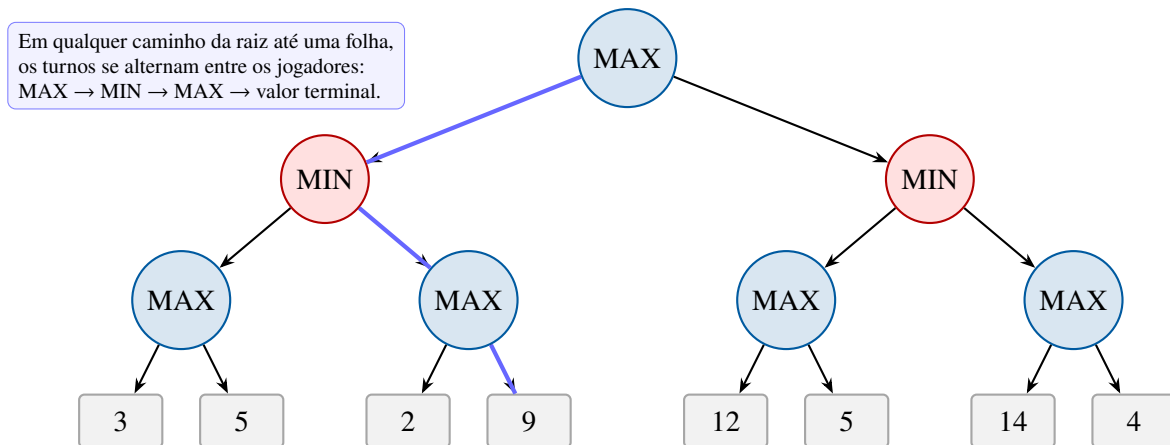


Figura 1: Árvore de jogo esquemática. A alternância entre jogadores ocorre ao longo de cada caminho da raiz até uma folha, refletindo a sucessão de turnos no jogo.

- Os níveis da árvore alternam entre jogadas de MAX e de MIN: no nível 0 (raiz) é a vez de MAX; no nível 1 é a vez de MIN; no nível 2, de MAX novamente; e assim por diante.
- Na árvore de jogo completa considerada pelo Minimax exato, cada nó folha representa um estado terminal e tem um valor numérico associado: sua utilidade, calculada pela função UTILITY.
- Em nós MAX, o algoritmo escolherá o maior valor entre os filhos; em nós MIN, assume-se que o oponente escolherá o menor.

A Figura 2 ilustra a estrutura geral de uma árvore de jogo com dois níveis de profundidade. Os círculos azuis representam estados em que é a vez de MAX jogar; os círculos vermelhos representam estados em que é a vez de MIN jogar; e os retângulos cinza exibem utilidades terminais vistas da perspectiva de MAX.

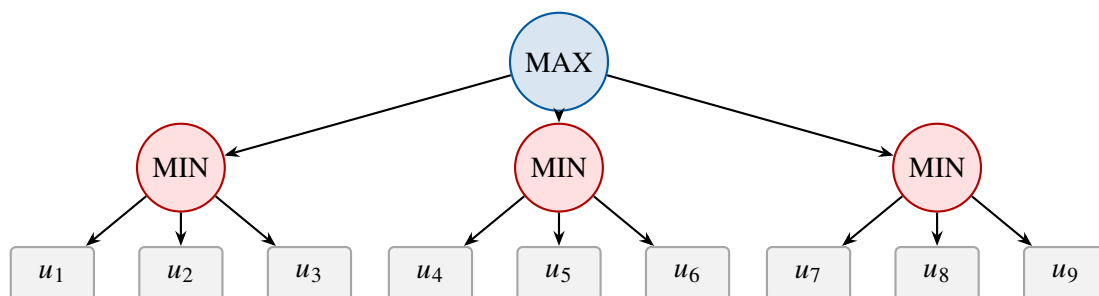


Figura 2: Estrutura básica de uma árvore de jogo. MAX escolhe uma ação na raiz, MIN responde no nível seguinte e as folhas armazenam utilidades $u_1, \dots, u_9$.

Os valores $u_1, \dots, u_9$ são calculados pela função UTILITY da perspectiva de MAX: valores altos são bons para MAX, valores baixos são bons para MIN.

4.2 Diferença em relação à árvore de busca monoagente

Na árvore de busca monoagente, o agente controla todas as decisões: ele escolhe qual aresta percorrer em cada nível. Na árvore de jogo, o agente controla apenas os nós MAX; nos nós MIN,

é o oponente quem decide. O agente precisa, portanto, construir uma estratégia que funcione bem *independentemente do que o oponente faça*.

Na prática, árvores de jogo completas são inviáveis de construir para jogos não triviais. É importante distinguir duas medidas: a *complexidade do espaço de estados* conta posições distintas possíveis, enquanto a *complexidade da árvore de jogo* conta sequências possíveis de jogadas. Mesmo em um jogo pequeno como o jogo da velha, a árvore completa sem fundir posições repetidas tem centenas de milhares de nós. No xadrez, a árvore de jogo de uma partida completa é tradicionalmente estimada em torno de 10^{120} – 10^{123} sequências possíveis. Essa explosão combinatória é precisamente o motivo pelo qual o Minimax exato só é viável em jogos pequenos. Voltaremos a esse ponto na seção sobre funções de avaliação.

Pergunta 5

Considere um caminho da raiz até uma folha em uma árvore de jogo. Por que MAX não pode escolher livremente todas as arestas desse caminho, como ocorreria em uma árvore de busca monoagente? Explique o papel dos nós MAX e MIN e como essa alternância afeta a forma de avaliar um caminho.

5 Valor Minimax

Para decidir qual ação tomar em um estado, o agente MAX precisa de um critério para comparar as possíveis ações (jogadas). Esse critério é o denominado *valor minimax* de um estado.

O valor minimax de um estado s , denotado $\text{MINIMAX}(s)$, é a utilidade que MAX pode *garantir* a partir de s , supondo que ambos os jogadores joguem de forma ótima até o final do jogo. Trata-se de uma análise de pior caso: MAX assume que MIN responderá de maneira ótima (i.e., da forma mais desfavorável possível para MAX).

A definição recursiva é:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{se } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{se } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{se } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

A lógica por trás dessa definição pode ser entendida assim:

- Nos estados terminais, o valor é simplesmente a utilidade daquele estado (conhecida pelas regras do jogo).
- Nos estados de MAX, ele escolhe a ação que maximiza o valor minimax do estado resultante.
- Nos estados de MIN, ele escolhe a ação que minimiza o valor minimax, pois MIN quer o pior resultado possível para MAX.

5.1 Exemplo resolvido: calculando valores minimax

Neste exemplo, vamos ilustrar o cálculo do valor minimax passo a passo. A Figura 3 apresenta uma raiz MAX com três ações possíveis, a_1 , a_2 e a_3 , cada uma levando a uma resposta de MIN e, em seguida, a utilidades terminais. A árvore foi deliberadamente construída com poucos níveis e folhas na mesma profundidade para tornar a propagação dos valores visível. Em jogos reais, a profundidade máxima pode ser muito maior, e diferentes sequências de jogadas podem atingir estados terminais em profundidades distintas; o Minimax aplica a mesma regra recursiva a cada nó, independentemente dessa diferença.

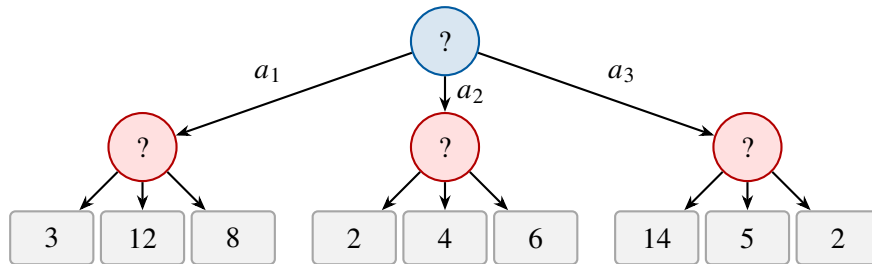


Figura 3: Árvore inicial do exemplo de Minimax. Os valores internos ainda são desconhecidos; as folhas contêm as utilidades terminais que serão propagadas de baixo para cima.

O algoritmo percorre a árvore em profundidade: segue cada ramo até uma folha e, no retorno da recursão, propaga os valores para cima. A ordem de visita e o raciocínio em cada passo:

Passo 1: Nó MIN esquerdo (ramo a_1):

- Visitamos o filho 1: utilidade = 3. Valor parcial do nó MIN: $v = 3$.
- Visitamos o filho 2: utilidade = 12. Como $\min(3, 12) = 3$, v permanece 3.
- Visitamos o filho 3: utilidade = 8. Como $\min(3, 8) = 3$, v permanece 3.
- **Resultado:** MINIMAX(nó MIN esquerdo) = 3.

Passo 2: Nó MIN central (ramo a_2):

- Visitamos o filho 1: utilidade = 2. Valor parcial: $v = 2$.
- Visitamos o filho 2: utilidade = 4. Como $\min(2, 4) = 2$, v permanece 2.
- Visitamos o filho 3: utilidade = 6. Como $\min(2, 6) = 2$, v permanece 2.
- **Resultado:** MINIMAX(nó MIN central) = 2.

Passo 3: Nó MIN direito (ramo a_3):

- Visitamos o filho 1: utilidade = 14. Valor parcial: $v = 14$.
- Visitamos o filho 2: utilidade = 5. Como $\min(14, 5) = 5$, v atualiza para 5.
- Visitamos o filho 3: utilidade = 2. Como $\min(5, 2) = 2$, v atualiza para 2.
- **Resultado:** MINIMAX(nó MIN direito) = 2.

Passo 4: Nó MAX raiz:

- Os três filhos têm valores 3, 2 e 2 (pelos ramos a_1 , a_2 , a_3).
- MAX escolhe o máximo: $\max(3, 2, 2) = 3$.
- **Resultado:** MINIMAX(raiz) = 3.

A Figura 4 mostra a mesma árvore após a propagação dos valores: cada nó MIN recebe o mínimo de suas folhas, e a raiz MAX recebe o máximo entre os valores dos três ramos.

Da decisão na raiz: O valor da raiz é 3, obtido pelo ramo a_1 . Portanto, a ação ótima de MAX no estado inicial é a_1 :

$$\pi^*(s_0) = a_1 = \arg \max_{a \in \{a_1, a_2, a_3\}} \text{MINIMAX}(\text{RESULT}(s_0, a)) = \arg \max(3, 2, 2)$$

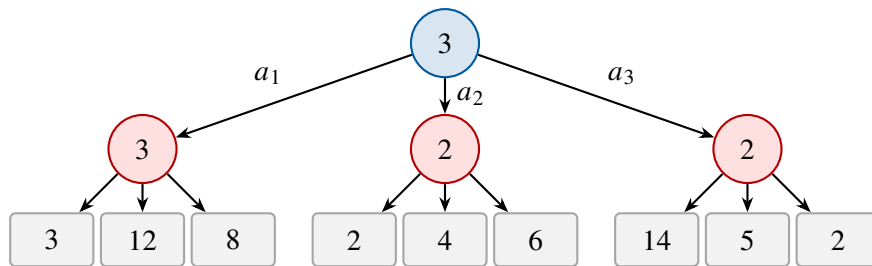


Figura 4: Árvore do exemplo de Minimax após a propagação dos valores. Os nós MIN têm valores 3, 2 e 2; a raiz MAX recebe valor 3 e seleciona a ação a_1 .

MAX escolhe o ramo cujo pior caso é o melhor. Uma vez nesse ramo (a_1), MIN, se jogar otimamente, escolherá o filho de menor utilidade, isto é, 3. MAX consegue garantir utilidade pelo menos 3, independentemente das escolhas de MIN. Se MIN também jogar otimamente, o resultado será exatamente 3.

Cuidado com uma confusão frequente: o fato de o ramo a_1 conter uma folha com utilidade 12 *não* significa que MAX possa garantir 12. O valor do ramo é 3, porque MIN sempre escolherá o pior caso para MAX. MAX pode obter 12 apenas se MIN cometer um erro.

Pergunta 6

O Minimax assume, ao calcular seus valores, que o oponente escolherá sempre a resposta mais desfavorável para MAX. Isso deve ser interpretado como uma limitação ou como uma garantia do algoritmo? Explique em que sentido o Minimax é conservador e o que pode acontecer quando MIN joga de forma subótima.

Pergunta 7

No exemplo acima, suponha que MIN não seja um jogador perfeito e, no nó MIN esquerdo, escolha a ação que leva à utilidade 12 em vez de 3. O resultado para MAX seria melhor ou pior do que o garantido pelo Minimax? O que isso diz sobre o comportamento do algoritmo quando o oponente comete erros?

6 Algoritmo Minimax

O algoritmo Minimax realiza uma exploração em profundidade (DFS) a partir do estado inicial. Durante essa exploração, cada ação considerada gera, por meio de RESULT, um novo estado sucessor; assim, a porção da árvore de jogo necessária à busca é construída progressivamente à medida que a execução avança. Quando o algoritmo alcança um estado terminal, calcula sua utilidade. No retorno das chamadas recursivas, os valores obtidos nos estados sucessores são combinados por operações de máximo ou mínimo e propagados em direção ao estado inicial.

6.1 Implementação

O Algoritmo 1 apresenta o pseudocódigo do Minimax. Esse algoritmo formaliza diretamente a definição recursiva MINIMAX(s) descrita na Seção 5. O pseudocódigo é dividido em três funções mutuamente recursivas, correspondendo aos três casos da definição: estado terminal, nó MAX e nó MIN.

O algoritmo funciona da seguinte forma. **VALUE** é o ponto de entrada: recebe um estado s e decide o que fazer com ele. Se s for terminal (linha 2), retorna sua utilidade diretamente. Caso contrário, delega para **MAX-VALUE** ou **MIN-VALUE** conforme o jogador ativo (linhas 5–8).

- **MAX-VALUE** (linhas 10–14) implementa o caso em que é a vez de **MAX**: itera sobre todas as ações legais em s , gera o estado sucessor com **RESULT**, chama **VALUE** recursivamente sobre esse sucessor e mantém o maior valor encontrado até o momento no acumulador v . Ao final, retorna o maior valor, que é exatamente o que **MAX** pode garantir a partir de s .
- **MIN-VALUE** (linhas 16–20) é simétrica: realiza o mesmo processo, mas mantém o *menor* valor encontrado, pois **MIN** quer o pior resultado possível para **MAX**.

A recursão mútua entre as três funções espelha diretamente a alternância de turnos na árvore de jogo: cada chamada a **VALUE** sobre um filho de um nó **MAX** resultará em uma chamada a **MIN-VALUE**, e vice-versa, até que se chegue a um estado terminal.

Algoritmo 1 Minimax

```

1: função VALUE( $s$ ):
2:   if TERMINAL-TEST( $s$ ) then
3:     return UTILITY( $s$ )
4:   end if
5:   if PLAYER( $s$ ) = MAX then
6:     return MAX-VALUE( $s$ )
7:   else
8:     return MIN-VALUE( $s$ )
9:   end if
10: função MAX-VALUE( $s$ ):
11:    $v \leftarrow -\infty$ 
12:   for cada  $a \in \text{ACTIONS}(s)$  do
13:      $v \leftarrow \max(v, \text{VALUE}(\text{RESULT}(s, a)))$ 
14:   end for
15:   return  $v$ 
16: função MIN-VALUE( $s$ ):
17:    $v \leftarrow +\infty$ 
18:   for cada  $a \in \text{ACTIONS}(s)$  do
19:      $v \leftarrow \min(v, \text{VALUE}(\text{RESULT}(s, a)))$ 
20:   end for
21:   return  $v$ 

```

6.2 Exemplo resolvido: execução recursiva do Minimax

Retomemos a árvore da Figura 3. Denote a raiz por s_0 e os estados alcançados pelas ações a_1 , a_2 e a_3 por s_1 , s_2 e s_3 , respectivamente. Assumimos que as ações e os filhos de cada nó são examinados da esquerda para a direita. A tabela a seguir mostra as chamadas principais durante a execução do Minimax; cada grupo de chamadas a folhas corresponde a uma descida em profundidade seguida pelo retorno recursivo ao nó **MIN** correspondente.

Passo	Chamada ou re-torno	Ação executada	Valor retornado / acumulado
1	VALUE(s_0)	s_0 não é terminal e $\text{PLAYER}(s_0) = \text{MAX}$	chama $\text{MAX-VALUE}(s_0)$
2	MAX-VALUE(s_0)	inicializa $v = -\infty$ e examina a_1	chama $\text{VALUE}(s_1)$
3	MIN-VALUE(s_1)	visita as folhas 3, 12 e 8 em profundidade	$\min(3, 12, 8) = 3$
4	MAX-VALUE(s_0)	recebe 3 de a_1	$v \leftarrow \max(-\infty, 3) = 3$
5	MIN-VALUE(s_2)	visita as folhas 2, 4 e 6 em profundidade	$\min(2, 4, 6) = 2$
6	MAX-VALUE(s_0)	recebe 2 de a_2	$v \leftarrow \max(3, 2) = 3$
7	MIN-VALUE(s_3)	visita as folhas 14, 5 e 2 em profundidade	$\min(14, 5, 2) = 2$
8	MAX-VALUE(s_0)	recebe 2 de a_3	$v \leftarrow \max(3, 2) = 3$
9	VALUE(s_0)	recebe o retorno de $\text{MAX-VALUE}(s_0)$	$\text{MINIMAX}(s_0) = 3$

Por exemplo, no passo 3, $\text{MIN-VALUE}(s_1)$ não calcula os três valores de uma só vez: ela chama VALUE para a folha com utilidade 3, recebe 3, atualiza seu acumulador para 3 e só então visita as folhas seguintes. Como todas as folhas são terminais, cada uma retorna imediatamente sua utilidade. A recursão retorna ao nó MAX somente depois de completar todo o ramo de MIN ; por isso, trata-se de uma busca em profundidade. Ao final, a_1 é a ação ótima na raiz, pois seu pior caso, 3, é maior que os piores casos 2 de a_2 e a_3 .

Sobre os valores iniciais $-\infty$ e $+\infty$. As funções MAX-VALUE e MIN-VALUE iniciam seus acumuladores com $-\infty$ e $+\infty$, respectivamente. Por que não inicializar simplesmente com o valor do primeiro filho? Porque precisamos que a inicialização não contamine a comparação desde o início. Se a utilidade mínima do jogo for, digamos, -100 , inicializar com -50 faria com que o acumulador não representasse corretamente o melhor valor encontrado até o momento. Usar $-\infty$ garante que qualquer utilidade real seja melhor do que o valor inicial.

A implementação do algoritmo Minimax pode ser expressa de forma compacta em Python. Na Listagem 1, $\text{successors}(\text{state})$ corresponde a gerar $\text{RESULT}(s, a)$ para cada ação $a \in \text{ACTIONS}(s)$; o restante da estrutura segue diretamente o pseudocódigo.

Observe que value é chamada recursivamente dentro de max_value e min_value , e essas por sua vez são chamadas de dentro de value : essa recursão mútua implementa a alternância de turnos. A profundidade máxima da pilha de chamadas é proporcional à profundidade da árvore m . A memória total também depende de como os sucessores são gerados, como discutido a seguir.

Observação sobre retorno de valor vs. ação: As funções acima retornam o *valor* da raiz, não a ação ótima. Para retornar também a ação, seria preciso armazenar o melhor sucessor no nível da raiz, como faremos na função ALPHA-BETA-SEARCH da próxima seção.

```

1 def value(state):
2     if terminal_test(state):
3         return utility(state)      # caso base: estado terminal
4     if player(state) == MAX:
5         return max_value(state)    # vez de MAX: maximiza

```

```
6     return min_value(state)          # vez de MIN: minimiza
7
8 def max_value(state):
9     v = -inf                          # pior valor possível para MAX
10    for s in successors(state):
11        v = max(v, value(s))          # propaga o maior valor filho
12    return v
13
14 def min_value(state):
15     v = +inf                          # pior valor possível para MIN
16    for s in successors(state):
17        v = min(v, value(s))          # propaga o menor valor filho
18    return v
```

Listagem 1: Implementação do algoritmo Minimax em Python.

6.3 Propriedades

- **Completo?** Sim, desde que a árvore de jogo seja finita.
- **Ótimo contra um oponente perfeito?** Sim. O Minimax produz a decisão ótima para MAX supondo que MIN também joga de forma ótima. Nenhum outro algoritmo pode garantir um resultado melhor contra um adversário racional.
- **Explora otimamente um oponente imperfeito?** Não necessariamente. O Minimax otimiza o pior caso e não modela erros específicos do adversário. Se conhecermos uma tendência sistemática de MIN a jogar de forma subótima, uma estratégia adaptada a esse comportamento pode obter utilidade esperada maior, ao custo de abandonar parte da garantia de pior caso. Em suma, o Minimax é seguro, mas conservador.
- **Complexidade de tempo:** $O(b^m)$, onde b é o fator de ramificação e m é a profundidade máxima da árvore.
- **Complexidade de espaço:** a análise tradicional é $O(bm)$ quando, em cada nível da DFS, os sucessores ainda não explorados são materializados e mantidos em memória. Se os sucessores forem gerados um a um, a memória auxiliar pode aproximar-se de $O(m)$, além da representação dos estados.

Tomando valores típicos para o xadrez ($b \approx 35$, $m \approx 80$ para uma partida completa), o número de nós a explorar é da ordem de $35^{80} \approx 10^{123}$, muito além de qualquer capacidade computacional. Isso torna a busca exata inviável para jogos não triviais, motivando as extensões que veremos a seguir.

Pergunta 8

O algoritmo Minimax tem complexidade de tempo $O(b^m)$, a mesma do DFS em uma busca monoagente. Por que as complexidades são idênticas? O que isso revela sobre a natureza da exploração realizada pelo Minimax? *Dica: como o Minimax trata a estrutura de dois jogadores em relação à estrutura da pilha de recursão?*

7 Poda Alfa-Beta

A poda alfa-beta (*alpha-beta pruning*) é uma otimização do Minimax que evita a expansão de partes da árvore de jogo que não podem influenciar a decisão final. Ela retorna *exatamente o mesmo resultado* que o Minimax completo, porém explorando potencialmente muito menos nós.

7.1 Ideia central

Durante a busca, o algoritmo mantém dois limites ao longo do caminho da raiz até o nó atual; eles não resumem toda a árvore, mas apenas as restrições já estabelecidas pelos ancestrais:

- α : o *melhor limite inferior* já estabelecido para MAX ao longo do caminho. MAX já conhece alguma alternativa que garante valor pelo menos α .
- β : o *melhor limite superior* já estabelecido para o valor de MAX ao longo do caminho, decorrente das escolhas de MIN. Ao longo do caminho atual, algum ancestral MIN já possui uma alternativa cujo valor para MAX é no máximo β .

Inicialmente, $\alpha = -\infty$ e $\beta = +\infty$. Quando a desigualdade $\alpha \geq \beta$ se torna verdadeira em um nó, continuar explorando o ramo não pode alterar a escolha de algum ancestral: MAX já tem uma alternativa pelo menos tão boa quanto aquilo que MIN permitiria nesse ramo. Por isso, os filhos restantes desse nó não precisam mais ser explorados.

Considere uma situação simples em que MAX já avaliou um primeiro ramo da árvore e, em seguida, começa a explorar um segundo ramo controlado por MIN. O objetivo do exemplo é mostrar como os limites α e β , construídos a partir dos valores já observados, podem tornar desnecessária a exploração completa desse segundo ramo. A Figura 5 representa um *instantâneo* dessa execução do Minimax com poda alfa-beta. A árvore mostrada não deve ser entendida como uma estrutura previamente construída: seus nós são gerados progressivamente à medida que a busca avança em profundidade. A figura destaca justamente o momento em que as informações acumuladas permitem concluir que parte da subárvore restante não pode alterar a decisão final. Para compreender como a execução chega a esse ponto, acompanhe a sequência descrita nos itens a seguir.

- Inicialmente, temos apenas a raiz (MAX), com $\alpha = -\infty$ e $\beta = +\infty$. A busca começa pelo ramo esquerdo. Ao expandir o nó MIN à esquerda, seus filhos são avaliados (valores 3 e 5), e portanto MIN devolve $\min(3, 5) = 3$ para a raiz. Nesse momento, MAX atualiza seu melhor valor conhecido: $\alpha \leftarrow 3$. Isso significa que MAX já sabe que pode garantir, no mínimo, o valor 3.
- A busca então prossegue para o ramo direito. Ao entrar no nó MIN da direita, esse nó herda a informação de que, acima dele, já vale $\alpha = 3$. Em seguida, a busca avalia o primeiro filho desse nó, obtendo o valor 2. Como MIN sempre escolhe o menor valor entre seus filhos, ele já sabe que poderá devolver, no máximo, 2 para seu pai; assim, atualiza-se $\beta \leftarrow 2$.
- Neste ponto ocorre a poda. No nó MIN da direita, temos $\alpha \geq \beta$, isto é, $3 \geq 2$. Isso indica que MAX já possui uma alternativa (o ramo esquerdo) que garante valor 3, enquanto o melhor que esse novo ramo pode oferecer é, no máximo, 2. Logo, mesmo sem explorar os filhos restantes desse nó, já é certo que MAX nunca escolherá esse ramo.
- Assim, a subárvore restante é descartada. A figura destaca exatamente esse momento: após a avaliação sequencial dos nós (da esquerda para a direita), a condição $\alpha \geq \beta$ torna explícito que continuar a busca nesse ramo não pode alterar a decisão final do minimax.

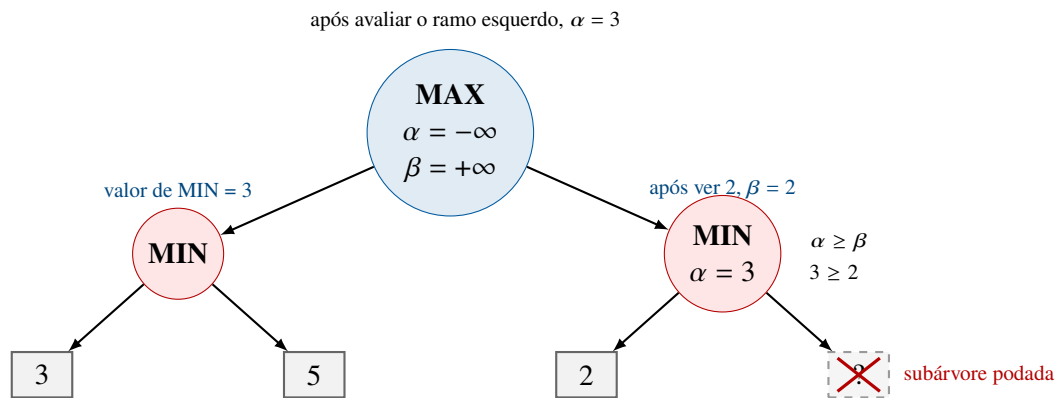


Figura 5: No ramo direito, assim que MIN encontra um filho com valor 2, obtemos $\beta = 2$. Como nesse nó vale $\alpha \geq \beta$ (isto é, $3 \geq 2$), os filhos restantes podem ser podados; mesmo que fossem avaliados, MIN jamais devolveria um valor maior que 2, e portanto MAX nunca preferiria esse ramo ao valor 3 já conhecido.

7.2 Por que a poda não altera o resultado

Antes do algoritmo formal, é essencial entender o argumento que garante a correção da poda.

- Poda em nó MIN: Suponha que estamos avaliando um nó MIN n , e o valor parcial acumulado de n cai abaixo de α . O ancestral de n é um nó MAX, que já tem garantida uma utilidade $\geq \alpha$ em outro ramo. Se o valor final de n for $\leq \alpha$, esse nó MAX nunca escolherá ir para n ; ele sempre preferirá o ramo onde α já foi obtido. Portanto, os filhos restantes de n não podem alterar a decisão do ancestral MAX. A condição de poda é $v \leq \alpha$.
- Poda em nó MAX: O raciocínio é simétrico. Se o valor parcial de um nó MAX supera β , o ancestral MIN (que garante $\leq \beta$ em outro ramo) nunca escolherá esse nó. A condição de poda é $v \geq \beta$.

A condição unificada é: sempre que $\alpha \geq \beta$ no nó atual, os filhos restantes são podados. A ordem de exploração não altera o valor retornado pelo algoritmo; ela altera apenas quantos nós precisam ser examinados.

Pergunta 9

Explique, sem recorrer apenas às fórmulas, o significado de α e β durante a busca alfa-beta. O que cada valor representa para MAX e para MIN, respectivamente?

7.3 Exemplo resolvido: poda alfa-beta passo a passo

Neste exemplo, vamos ilustrar a execução da poda alfa-beta com rastreamento de α e β . A Figura 6 mostra a árvore usada: os ramos a_2 e a_3 têm filhos marcados com $\times$, pois serão podados. Usaremos uma árvore baseada no exemplo anterior, mantendo os dois primeiros ramos e alterando os valores do terceiro ramo para 1, 5, 9. Percorreremos os filhos da esquerda para a direita; com essa alteração, o primeiro filho do terceiro nó MIN já aciona a poda imediatamente.

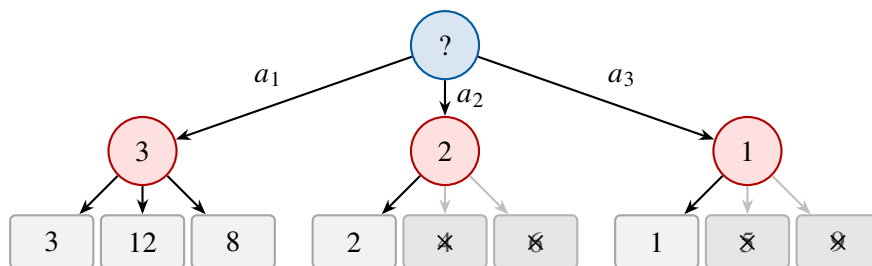


Figura 6: Execução esquemática da poda alfa-beta. Depois de avaliar o primeiro filho dos ramos a_2 e a_3 , os demais filhos desses nós MIN são podados, pois seus valores não podem alterar a decisão já garantida na raiz.

P.	Nó visitado	α gente	vi- gente	β gente	vi- gente	v	Observação
1	Raiz MAX – início	$-\infty$		$+\infty$		$-\infty$	Chama Min-Value(a_1)
2	MIN a_1 – filho 1 (u=3)	$-\infty$		3		3	$\beta \leftarrow 3$
3	MIN a_1 – filho 2 (u=12)	$-\infty$		3		3	$\min(3, 12) = 3$; β inalterado
4	MIN a_1 – filho 3 (u=8)	$-\infty$		3		3	$\min(3, 8) = 3$; β inalterado
5	Retorna à raiz: $v = 3$	3		$+\infty$		3	$\alpha \leftarrow 3$
6	MIN a_2 – filho 1 (u=2)	3		$+\infty$		2	$v = 2 \leq \alpha = 3$: poda!
	MIN a_2 – filhos 2, 3	3		$+\infty$		n/a	Podados
7	Retorna à raiz: $v = 2$	3		$+\infty$		3	$\max(3, 2) = 3$; α inalterado
8	MIN a_3 – filho 1 (u=1)	3		$+\infty$		1	$v = 1 \leq \alpha = 3$: poda!
	MIN a_3 – filhos 2, 3	3		$+\infty$		n/a	Podados
9	Retorna à raiz: $v = 1$	3		$+\infty$		3	Final: $\max(3, 2, 1) = 3$

Resultado: MINIMAX(raiz) = 3, idêntico ao Minimax completo. A decisão de MAX é a_1 .

Folhas avaliadas: utilidades 3, 12, 8 (MIN a_1 , todas avaliadas); 2 (MIN a_2 , apenas a primeira); 1 (MIN a_3 , apenas a primeira). Total: 5 folhas avaliadas em vez de 9.

Por que a poda no Passo 6 é válida? O nó MIN a_2 encontrou um filho com utilidade 2. O nó MAX raiz tem $\alpha = 3$ garantido pelo ramo a_1 . Mesmo que os filhos restantes do nó MIN a_2 produzissem valores enormes, o nó MIN escolheria $\min(2, \dots) \leq 2 < 3 = \alpha$. O nó MAX raiz nunca escolheria esse ramo. Avaliar os filhos restantes é computacionalmente inútil.

Por que a poda no Passo 8 é válida? Raciocínio idêntico: o nó MIN a_3 já encontrou utilidade 1, que é menor que $\alpha = 3$. Independentemente dos filhos restantes, este ramo não será escolhido por MAX.

Pergunta 10

A poda alfa-beta retorna o mesmo resultado que o Minimax sem poda, mesmo sem examinar todos os nós. Baseando-se no argumento da Seção 7.2 e no Exemplo 2, explique em suas próprias palavras: qual é a propriedade fundamental dos nós podados que garante que sua não-expansão não altera o resultado? Sua resposta deve mencionar explicitamente os papéis de α e β .

7.4 Algoritmo

O pseudocódigo apresentado no Algoritmo 2 estende o Minimax com os dois parâmetros adicionais α e β , que são herdados e atualizados ao longo da recursão. A estrutura geral permanece idêntica ao Minimax original: a diferença está nas verificações de poda inseridas dentro de cada laço.

A função ALPHA-BETA-SEARCH (linha 1) é o ponto de entrada: ela inicia a busca com $\alpha = -\infty$ e $\beta = +\infty$, indicando que ainda não há limites úteis estabelecidos. Em vez de retornar apenas o valor, ela percorre explicitamente os filhos da raiz, mantém o melhor limite inferior encontrado em α e armazena a ação correspondente em *best_action*. Assim, os limites alfa-beta aprendidos ao avaliar um ramo da raiz podem ser usados na avaliação dos ramos seguintes.

MAX-VALUE itera sobre os filhos e, após atualizar v com o valor retornado por MIN-VALUE, verifica a condição de poda: se $v \geq \beta$, MAX já encontrou um valor que o ancestral MIN jamais escolheria (pois MIN garante $\leq \beta$ em outro ramo), e os filhos restantes são abandonados. Se não houver poda, α é atualizado para refletir o melhor resultado garantido por MAX até o momento nesse caminho.

MIN-VALUE é simétrica: verifica $v \leq \alpha$ e atualiza β . A passagem de α e β por parâmetro mantém cada chamada recursiva com os limites herdados corretamente de seus ancestrais. Os valores obtidos em um ramo podem atualizar os limites de seus ancestrais e, assim, permitir podas em ramos irmãos; alterações locais em uma chamada, porém, não modificam indevidamente o contexto de chamadas anteriores.

Sobre o que é retornado: ALPHA-BETA-SEARCH retorna a ação ótima, enquanto as funções auxiliares retornam o valor. A implementação Python abaixo retorna apenas o valor da raiz; para retornar a ação, basta usar a mesma ideia do pseudocódigo e armazenar o melhor sucessor no nível da raiz.

O código Python abaixo apresenta a mesma recursão alfa-beta, mas retorna apenas o valor da raiz. Os parâmetros alpha e beta são passados a cada chamada recursiva e atualizados localmente; os retornos antecipados (return v após a verificação de poda) correspondem às linhas de poda do pseudocódigo.

A Listagem 2 apresenta uma implementação recursiva do algoritmo de poda alfa-beta. Ela retorna apenas o valor da raiz, mas usa os mesmos limites α e β do pseudocódigo: a poda ocorre quando esses limites mostram que os sucessores restantes não podem alterar a decisão já determinada pelos ancestrais.

```

1  # Retorna o valor da raiz. Para retornar a acao otima,
2  # armazene o melhor sucessor no nivel da raiz (ver Alpha-Beta-Search
   acima).
3
4  def alpha_beta_search(state):
5      return max_value(state, -inf, +inf) # inicia com limites sem
      restricao
6
7  def max_value(state, alpha, beta):
8      if terminal_test(state):
9          return utility(state)
10     v = -inf
11     for s in successors(state):
12         v = max(v, min_value(s, alpha, beta))
13         if v >= beta:
14             return v # poda beta: MIN nunca escolheria este

```

```

15         ramo
16         alpha = max(alpha, v) # atualiza o limite inferior de MAX
17     return v
18
19 def min_value(state, alpha, beta):
20     if terminal_test(state):
21         return utility(state)
22     v = +inf
23     for s in successors(state):
24         v = min(v, max_value(s, alpha, beta))
25         if v <= alpha:
26             return v # poda alfa: MAX nunca escolheria este
27             ramo
28         beta = min(beta, v) # atualiza o limite superior para MAX
29     return v

```

Listagem 2: Poda Alfa-Beta em Python

7.5 Eficiência e ordem de varredura

A eficiência da poda alfa-beta depende crucialmente da *ordem em que os filhos são examinados*. A ordem de exploração não altera o valor retornado pelo algoritmo; ela altera apenas quantos nós precisam ser examinados:

- **Caso ótimo** (melhor ordenação): a poda precisa examinar apenas $O(b^{m/2})$ nós, reduzindo o fator de ramificação efetivo de b para $\sqrt{b}$. Para o xadrez, isso significa que, com o mesmo tempo de computação, é possível explorar o *dobro* da profundidade em relação ao Minimax sem poda.
- **Caso médio** (ordenação aleatória): a poda examina $O(b^{3m/4})$ nós (valor de referência analítico aproximado).
- **Caso pessimista** (pior ordenação): nenhuma poda ocorre e o desempenho é idêntico ao Minimax sem poda.

Isso sugere o uso de *heurísticas de ordenação*, como examinar capturas antes de movimentos passivos no xadrez, para aproximar o desempenho do caso ótimo.

Pergunta 11

A eficiência da poda alfa-beta depende da ordem em que os filhos são examinados. Explique por que avaliar primeiro as jogadas mais promissoras tende a produzir mais podas. No xadrez, que tipos de heurísticas poderiam ser usados para ordenar os movimentos de modo a aumentar a probabilidade de poda? Considere separadamente o que seria desejável em nós MAX e em nós MIN.

Repare que a poda alfa-beta não compromete a otimalidade do Minimax: ela sempre retorna a mesma decisão que o Minimax completo retornaria. Observe que Alfa-Beta não muda a resposta do Minimax; ele apenas reduz o trabalho necessário para obtê-la. Os nós podados são exatamente aqueles que, por construção, não poderiam alterar o valor da raiz.

Pergunta 12

A poda alfa-beta consegue, no melhor caso, reduzir a complexidade de $O(b^m)$ para $O(b^{m/2})$. Para o xadrez ($b \approx 35$, 100 segundos por jogada, 10K nós/s):

- (a) Calcule quantos nós o agente pode visitar por jogada.
- (b) Com o Minimax sem poda, a que profundidade d ele chega, sabendo que $35^d \leq 10^6$?
- (c) Com a poda alfa-beta no melhor caso, a que profundidade ele chega, sabendo que $35^{d/2} \leq 10^6$?
- (d) O que essa diferença de profundidade representa em termos de qualidade do jogo?

8 Funções de Avaliação Heurística

Como vimos, o Minimax com poda alfa-beta ainda tem complexidade exponencial na profundidade da árvore. Para jogos como xadrez ou Go, é simplesmente impossível expandir a árvore até os estados terminais dentro de qualquer limite de tempo razoável.

A solução clássica para esse problema foi proposta por Claude Shannon em 1950: em vez de buscar necessariamente até os estados terminais, realiza-se uma busca com profundidade limitada (DLS). Quando a busca encontra um estado terminal, usa sua utilidade exata; quando para em um estado não terminal por causa do limite de profundidade, aplica uma *função de avaliação heurística* que estima o valor daquele estado. O algoritmo resultante é frequentemente chamado de *H-Minimax* (*heuristic minimax*).

Por que buscar profundo antes de avaliar? Uma dúvida natural: se temos uma boa função de avaliação EVAL, por que não simplesmente olhar um passo à frente e escolher o filho com maior EVAL, sem nenhuma busca? A resposta é que EVAL é sempre uma *estimativa imperfeita*. Ao buscar mais profundamente, o agente pode descobrir *consequências reais* das jogadas que EVAL sozinha não consegue ver. Um estado que parece ótimo segundo EVAL pode esconder uma armadilha dois lances à frente.

Em geral, maior profundidade tende a produzir decisões melhores, pois permite revelar consequências que estavam além do horizonte; isso, porém, não constitui uma garantia monotônica quando a avaliação é heurística. Há casos em que uma avaliação imperfeita pode induzir decisões piores em uma profundidade maior.

Uma função de avaliação *perfeita* $EVAL(s) = UTILITY^*(s)$ tornaria desnecessária qualquer busca em profundidade: bastaria escolher o filho com maior EVAL. Mas calcular tal função perfeita é computacionalmente tão difícil quanto fazer a busca completa: para saber o resultado exato de um estado, precisaríamos resolver o jogo a partir dele. Na prática, usamos funções imperfeitas e, em geral, maior profundidade ajuda a compensar suas limitações. Em resumo: *EVAL orienta; a busca em profundidade confirma ou corrige*.

8.1 A ideia

No H-Minimax, o algoritmo testa primeiro se o estado é terminal. Caso não seja, aplica também um *teste de corte*, que interrompe a expansão quando algum limite de busca, como a profundi-

dade máxima, é atingido. Ao atingir uma folha da busca:

- se o estado é terminal, utiliza-se $UTILITY(s)$;
- se o estado não é terminal e a busca foi interrompida pelo limite de profundidade, utiliza-se $EVAL(s)$.

É importante distinguir: no Minimax exato, as folhas da busca são estados terminais e seus valores são exatos. No H-Minimax, as folhas também podem ser estados não terminais onde a busca foi interrompida pelo teste de corte; nesses estados, $EVAL$ fornece apenas uma estimativa do valor real.

Com essas modificações, o agente pode tomar uma decisão em tempo limitado. A qualidade da decisão depende de duas coisas: a profundidade da busca e a qualidade da função de avaliação.

Com funções de avaliação e busca com profundidade limitada, a garantia de otimalidade do Minimax desaparece. O agente pode tomar decisões subótimas, pois os valores computados nos nós folha são apenas estimativas. Quanto melhor a função de avaliação e quanto maior a profundidade alcançada, mais próximo do ótimo tende a ser o resultado na prática.

Pergunta 13

Distinga $UTILITY$ de $EVAL$. Em seguida, explique a diferença entre um estado terminal e uma folha produzida pelo teste de corte no H-Minimax.

8.2 Requisitos de uma boa função de avaliação

Uma boa função de avaliação deve satisfazer três requisitos fundamentais:

- **Rapidez de computação:** a função é chamada para cada nó folha da busca, o que pode ocorrer milhões de vezes por segundo. Uma função lenta reduz drasticamente a profundidade alcançável.
- **Correlação com o resultado real:** a função deve produzir estimativas correlacionadas com o resultado do jogo a partir daquele estado.
- **Capacidade de ranquear corretamente estados próximos:** mesmo que os valores absolutos não sejam exatos, a ordenação induzida por $EVAL$ deve concordar, tão frequentemente quanto possível, com a ordenação determinada pelos valores reais dos estados.

8.3 Funções de avaliação lineares ponderadas

Uma forma clássica e simples combina **características** (*features*) do estado de forma linear:

$$EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_k f_k(s) = \sum_{i=1}^k w_i f_i(s)$$

Cada $f_i(s)$ mede um aspecto do estado que se acredita ser relevante para a qualidade da posição, e cada w_i é um peso que indica a importância relativa dessa característica. Note que muitas características são naturalmente definidas como a diferença entre MAX e MIN: isso captura a vantagem relativa de um jogador sobre o outro.

Exemplo: xadrez

No xadrez, características típicas incluem:

- **Vantagem material:** diferença no valor total das peças entre brancas e pretas, usando pesos padrão (peão = 1, cavalo/bispo = 3, torre = 5, rainha = 9).
- **Estrutura de peões:** formações (dobrados, isolados, passados).
- **Segurança do rei:** cobertura das casas ao redor do rei.
- **Controle do centro:** diferença no controle de casas centrais.
- **Mobilidade:** diferença no número de movimentos legais disponíveis.

Uma função proposta por Alan Turing era simplesmente $EVAL(s) = w(s)/b(s)$, onde $w(s)$ é a soma dos valores das peças brancas e $b(s)$ a soma das pretas. O Deep Blue (IBM), que derrotou Garry Kasparov em 1997, usava uma função de avaliação com mais de 8.000 características. Motores modernos como o Stockfish combinam tabelas posicionais refinadas com redes neurais eficientemente atualizáveis (NNUE, *Efficiently Updatable Neural Network*).

8.4 Exemplo 1

Para ilustrar o papel de funções de avaliação heurística em jogos adversariais, consideremos o cenário simplificado mostrado na Figura 7. O agente (Pacman) deve coletar pílulas enquanto evita o fantasma.

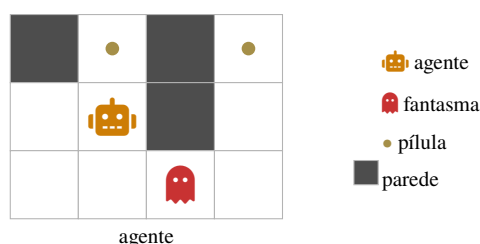


Figura 7: Estado inicial do ambiente usado no exemplo trabalhado da função de avaliação heurística. O agente deve aproximar-se das pílulas, mas a presença do fantasma pode tornar perigosa a região da comida mais próxima.

Suponha que a função de avaliação seja definida como:

$$EVAL(s) = score(s) + \frac{1}{d(A, F^*) + 1} - \frac{1}{d(G, F^*) + 1},$$

onde $score(s)$ representa o progresso já obtido pelo agente (por exemplo, o número de pílulas consumidas), A representa a posição do agente, G a posição do fantasma, e F^* é a posição da pílula ainda disponível mais próxima do agente. A função $d(\cdot, \cdot)$ representa a distância (por exemplo, Manhattan) no labirinto.

Essa função captura três intuições simples:

- Quanto maior o progresso já realizado pelo agente, melhor o estado;
- Quanto mais próximo o agente está da pílula mais próxima, melhor o estado;
- Quanto mais próximo o fantasma está dessa mesma pílula, pior o estado, pois o caminho até ela se torna perigoso.

Estado inicial. No estado inicial, o agente ainda não consumiu pílulas, está a uma unidade de distância da pílula mais próxima, e o fantasma está a três unidades dessa mesma pílula. Assim:

$$\text{EVAL}(s) = 0 + \frac{1}{1+1} - \frac{1}{3+1} = \frac{1}{2} - \frac{1}{4} = 0.25.$$

Após mover para cima. Se o agente se move para cima, ele alcança diretamente uma pílula e a consome. Essa pílula deixa de fazer parte do estado; portanto, F^* passa a ser a pílula restante mais próxima. Para manter o cálculo simples, suponha um estado sucessor ilustrativo em que o agente tenha agora $\text{score}(s) = 1$, esteja a duas unidades da pílula restante e o fantasma esteja a três unidades dela:

$$\text{EVAL}(s) = 1 + \frac{1}{2+1} - \frac{1}{3+1} \approx 1.083.$$

Esse valor mais alto reflete o ganho imediato obtido ao consumir a pílula.

Após mover para a esquerda. Se o agente se move para a esquerda, ele se afasta da pílula mais próxima. A distância passa a ser $d(A, F^*) = 2$, enquanto a posição do fantasma permanece a mesma:

$$\text{EVAL}(s) = \frac{1}{3} - \frac{1}{4} \approx 0.083.$$

Assim, esse movimento é claramente inferior ao movimento para cima.

Resposta do fantasma. Após o movimento do agente para cima, o fantasma pode responder de diferentes formas. Se ele se afasta, o valor do estado aumenta ligeiramente, pois a ameaça diminui. Por exemplo:

$$\text{EVAL}(s) = 1 + \frac{1}{3} - \frac{1}{5} = 1.133.$$

Por outro lado, se o fantasma se aproxima da pílula mais próxima do agente, o estado torna-se mais perigoso:

$$\text{EVAL}(s) = 1 + \frac{1}{3} - \frac{1}{3} \approx 1.0.$$

Observe que, mesmo nesse caso, a função de avaliação ainda atribui um valor relativamente alto ao estado. Isso ocorre porque a função considera apenas distâncias e não leva em conta consequências táticas mais profundas.

Limitações da função de avaliação. Esse exemplo ilustra uma limitação importante: duas posições podem receber valores semelhantes, mesmo que uma delas esteja prestes a levar a uma situação muito desfavorável (como uma colisão com o fantasma).

Funções heurísticas simples capturam apenas aspectos locais do estado e podem ignorar eventos iminentes que estão além do horizonte de busca. Essa limitação motiva o uso de técnicas adicionais, como a busca quiescente, que serão discutidas mais adiante.

8.5 Exemplo 2

Neste exemplo, vamos ilustrar o uso da função EVAL para comparar estados no jogo da velha. Definimos a seguinte função de avaliação para o jogo da velha, da perspectiva de MAX (X):

$$\text{EVAL}(s) = 3 \cdot f_1(s) + 2 \cdot f_2(s) - 3 \cdot f_3(s)$$

onde (todos os contadores são não-negativos):

- $f_1(s)$ = # linhas/colunas/diagonais com **2 X e 0 O** (ameaça imediata de MAX)
- $f_2(s)$ = # linhas/colunas/diagonais com **1 X e 0 O** (desenvolvimento de MAX)
- $f_3(s)$ = # linhas/colunas/diagonais com **2 O e 0 X** (ameaça imediata de MIN)

Note que o sinal negativo em $-3 \cdot f_3$ captura o fato de que ameaças de MIN são ruins para MAX. Todas as features são contagens positivas; o sinal é parte da fórmula.

Estado s_A : MAX (X) ocupa (1, 1) e (2, 2); MIN (O) ocupa (1, 3).

X	.	O
.	X	.
.	.	.

- $f_1(s_A)$: diagonal principal $\{(1, 1), (2, 2), (3, 3)\}$ tem 2 X e 0 O. $f_1 = 1$.
- $f_2(s_A)$: linha 2, coluna 1 e coluna 2 têm 1 X e 0 O. $f_2 = 3$.
- $f_3(s_A)$: nenhuma linha/col./diag. tem 2 O e 0 X. $f_3 = 0$.

$$\text{EVAL}(s_A) = 3(1) + 2(3) - 3(0) = 3 + 6 - 0 = \mathbf{9}$$

Estado s_B : MAX (X) ocupa (2, 2); MIN (O) ocupa (1, 1) e (1, 3).

O	.	O
.	X	.
.	.	.

- $f_1(s_B)$: nenhuma linha com 2 X e 0 O. $f_1 = 0$.
- $f_2(s_B)$: linha 2 e coluna 2 têm 1 X e 0 O. $f_2 = 2$.
- $f_3(s_B)$: linha 1 $\{(1, 1), (1, 2), (1, 3)\}$ tem 2 O e 0 X. $f_3 = 1$.

$$\text{EVAL}(s_B) = 3(0) + 2(2) - 3(1) = 0 + 4 - 3 = \mathbf{1}$$

Tabela resumo:

Estado	f_1	f_2	f_3	EVAL
s_A (X em (1, 1), (2, 2); O em (1, 3))	1	3	0	9
s_B (X em (2, 2); O em (1, 1), (1, 3))	0	2	1	1

Decisão: $\text{EVAL}(s_A) = 9 > \text{EVAL}(s_B) = 1$. O agente prefere s_A .

Conexão com a busca limitada: Em uma busca H-Minimax com profundidade máxima d , valores como esses seriam usados para estimar os nós folha não terminais. O Minimax propagaria esses valores para cima e escolheria a ação que maximiza o EVAL mínimo garantido contra MIN.

Pergunta 14

Se $\text{EVAL}(s_A) > \text{EVAL}(s_B)$, podemos concluir que s_A é necessariamente vencedor e s_B necessariamente perdedor? Explique o que a função de avaliação permite afirmar e o que ela não permite.

8.6 Funções de avaliação não lineares

Uma soma ponderada atribui a cada característica uma contribuição aditiva com peso fixo. Assim, ela não representa diretamente situações em que o efeito de uma característica depende do valor de outra, a menos que essas interações sejam explicitamente incluídas na representação. Por exemplo, no xadrez, a importância da segurança do rei depende da fase da partida: uma exposição do rei pode ser crítica no meio-jogo, mas muito menos problemática em determinados finais. Esse é um dos motivos pelos quais motores modernos recorrem a modelos mais sofisticados do que simples somas ponderadas.

Pergunta 15

Por que uma função de avaliação linear pode ser insuficiente para capturar toda a riqueza estratégica de um jogo como o xadrez? Construa um exemplo concreto: descreva dois estados de xadrez com os *mesmos valores individuais de características* (mesma contagem de peças, mesma mobilidade, etc.) mas onde um é estrategicamente muito superior ao outro. O que sua função linear faria nesses dois estados?

8.7 Aprendizado de funções de avaliação

Funções de avaliação podem ser projetadas manualmente por especialistas ou *aprendidas automaticamente* por meio de técnicas de aprendizado de máquina. O AlphaGo (DeepMind, 2016) usava redes neurais profundas treinadas tanto em partidas humanas quanto por auto-jogo (*self-play*). O AlphaZero (2017) foi além: sem nenhum conhecimento humano além das regras do jogo, aprendeu por auto-jogo, sem depender de partidas humanas como fonte principal de supervisão. Esses tópicos serão aprofundados quando estudarmos aprendizado de máquina.

9 A Importância da Profundidade

Em geral, quanto maior a profundidade que o agente consegue explorar, melhor tende a ser a qualidade de suas decisões, por duas razões complementares: (1) funções de avaliação são sempre imperfeitas, e mais profundidade pode reduzir o impacto dos erros de estimativa; (2) erros táticos só são detectados se a busca for profunda o suficiente para revelar suas consequências. Essa melhora, porém, não é monotonicamente garantida quando a avaliação das folhas é heurística.

Para ilustrar, considere um estado no jogo da velha que parece vantajoso para MAX a profundidade 1 (EVAL alto), mas onde MIN tem um fork disponível a profundidade 3, uma jogada que cria duas ameaças simultâneas de vitória, impossíveis de bloquear ao mesmo tempo. Uma busca com $d = 1$ não detecta esse problema; uma busca com $d = 3$ o detecta e escolhe um caminho diferente.

Ainda assim, mesmo profundidades razoáveis não eliminam completamente erros de avaliação no horizonte, o que motiva a *busca quiescente*.

10 Busca Quiescente

Mesmo com uma boa função de avaliação e profundidade razoável, a busca com profundidade limitada pode cometer erros graves por um motivo específico: o *efeito de horizonte* (*horizon effect*).

10.1 O efeito de horizonte

Suponha que a busca seja interrompida em um estado em que as pretas têm vantagem de material (um cavalo a mais). A função de avaliação declara esse estado como favorável para as pretas; o valor EVAL é positivo. Mas o próximo movimento das brancas captura a rainha preta sem compensação. A busca parou exatamente *antes* de revelar essa consequência crítica.

Um exemplo concreto no xadrez: imagine que a posição no horizonte ($d = 4$) mostra as pretas com um bispo a mais. $EVAL = +3$ (vantagem para as pretas). Mas as brancas, no lance seguinte (fora do horizonte), têm um tático de garfo de cavalo que captura tanto a rainha quanto a torre pretas. O agente, não enxergando esse lance, escolhe continuar sem se defender, e perde enormemente.

10.2 A solução: busca quiescente

A solução clássica é a busca quiescente (*quiescence search*): ao atingir o limite de profundidade, a função de avaliação só é aplicada em posições quiescentes, i.e., posições em que é improvável haver grandes variações de valor nas jogadas imediatas, isto é, posições livres de eventos táticos bruscos. Em jogos como xadrez, esses eventos incluem sobretudo capturas e, dependendo da implementação, xeques, promoções e outras ameaças forçantes.

A busca quiescente não resolve todos os erros de avaliação no horizonte, mas reduz fortemente os mais grosseiros.

Na prática, a busca quiescente pode adicionar várias camadas além do limite de profundidade nominal, especialmente em posições táticas complexas. Em implementações práticas, ela é tipicamente restrita a um subconjunto de movimentos taticamente relevantes, sobretudo capturas e, dependendo da implementação, xeques, promoções e outras ameaças forçantes. Isso limita seu custo computacional enquanto ainda elimina os erros mais grosseiros decorrentes do efeito de horizonte.

Exemplo: Xadrez. Uma posição pode ser considerada *não quiescente* quando há movimentos táticos imediatos que a implementação decidiu tratar, como capturas, xeques, promoções ou outras ameaças forçantes. Nessas situações, a busca é estendida além do limite de profundidade normal, restrita a esses movimentos “urgentes”, até que uma posição quiescente (estável) seja alcançada.

Exemplo: Pacman. A Figura 8 ilustra o efeito de horizonte em um ambiente simplificado inspirado no jogo Pacman. A árvore segue o modelo clássico de minimax, em que os jogadores alternam suas jogadas: no nível $d = 1$, o agente (MAX) atua; no nível $d = 2$, o fantasma (MIN) responde; e assim sucessivamente.

A busca é inicialmente limitada a uma profundidade fixa $d = 2$, indicada pela linha tracejada que representa o *horizonte de busca*. Em uma busca convencional (sem extensão quiescente), todos os nós nesse nível seriam avaliados diretamente pela função heurística.

Considere o ramo esquerdo da árvore. No nível $d = 2$, o nó à direita (marcado em vermelho) corresponde a uma situação não quiescente: o agente está adjacente ao fantasma, caracterizando um estado taticamente instável. No entanto, a função de avaliação retorna um valor relativamente alto ($EVAL = +0.87$), sugerindo que o estado ainda é favorável ao agente.

Esse valor é deliberadamente enganoso. Ele foi ajustado para fins didáticos, a fim de ilustrar que uma função heurística pode atribuir uma avaliação aparentemente razoável a um estado que, na verdade, esconde uma consequência adversa imediata. Em particular, a função de avaliação

considera apenas aspectos locais (como proximidade de comida e distância ao fantasma) e não captura adequadamente a iminência de uma colisão.

Dados esses valores, o fantasma (MIN) escolhe a ação que minimiza a avaliação do agente. Comparando os valores dos dois sucessores de MAX no ramo esquerdo (+0.90 e +0.87), o fantasma seleciona o nó não quiescente com $EVAL = +0.87$. No entanto, essa avaliação pode não ser confiável, pois ela oculta uma consequência adversa imediata.

A busca quiescente atua exatamente nesses casos. Em vez de interromper a expansão no nó não quiescente, o algoritmo identifica sua instabilidade e realiza uma extensão seletiva da busca a partir desse nó, indicada pela aresta tracejada. Essa extensão revela um desfecho crítico (colisão com o fantasma), levando a uma avaliação muito negativa ($EVAL = -500$).

Com essa correção, o valor do ramo esquerdo passa a refletir adequadamente o risco associado à jogada. Em particular, o valor propagado por MIN torna-se fortemente negativo, o que pode alterar a decisão de MAX na raiz.

Por contraste, o ramo à direita da raiz leva a um estado considerado quiescente. Nesse caso, não há ameaças imediatas, e o fantasma se afasta do agente, tornando a posição mais estável. A avaliação heurística ($EVAL = 0$) pode, portanto, ser considerada confiável nesse ponto, e a expansão é interrompida normalmente.

Em resumo, a busca quiescente modifica o critério de parada da busca adversarial: em vez de interromper a expansão estritamente na profundidade fixa, o algoritmo continua a expandir apenas os nós que representam estados não quiescentes, até atingir posições estáveis. Isso reduz o impacto do *horizon effect* e melhora a qualidade das decisões tomadas pelo algoritmo.

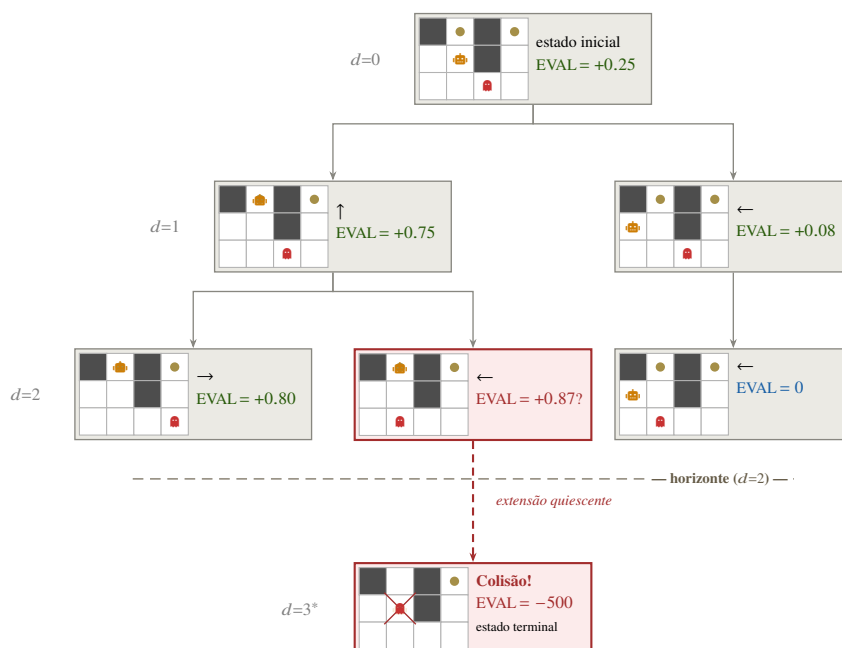


Figura 8: Exemplo de busca quiescente em uma árvore adversarial com alternância de turnos (MAX e MIN). A avaliação heurística considera a proximidade do agente à comida mais próxima e penaliza a proximidade do fantasma a essa mesma comida. No horizonte ($d = 2$), o nó em vermelho é não quiescente: embora receba uma avaliação aparentemente favorável ($EVAL = +0.87$), esse valor é enganoso e foi ajustado para fins didáticos, pois ignora uma consequência adversa imediata. A extensão quiescente revela uma colisão no nível seguinte ($EVAL = -500$), corrigindo a avaliação desse ramo.

Pergunta 16

Por que aplicar a função de avaliação em uma posição “não quiescente” (por exemplo, em que uma captura está disponível) pode levar a erros graves? Como a busca quiescente resolve esse problema sem explorar a árvore inteira até os estados terminais? Use o exemplo concreto da Seção 10.1 para guiar sua resposta.

11 Resumo

A tabela abaixo sintetiza as principais características do algoritmo Minimax e suas variantes. Note que Alfa-Beta não muda a resposta do Minimax; ele apenas reduz o trabalho necessário para obtê-la.

Variante	Compl.?	Ótimo?	Exato?	Tempo	Quando usar?	Espaço
Minimax	Sim [†]	Sim*	Sim	$O(b^m)$	Jogos pequenos / análise teórica	$O(bm)^{\ddagger}$
$\alpha\beta$ (melhor caso)	Sim [†]	Sim*	Sim	$O(b^{m/2})$	Prática com boa ordenação	$O(bm)^{\ddagger}$
$\alpha\beta$ (caso médio)	Sim [†]	Sim*	Sim	$O(b^{3m/4})$	Prática com ordenação aleatória	$O(bm)^{\ddagger}$
H-Minimax (+EVAL)	n/a	Não	Não	$O(b^d)$	Jogos reais com limite de tempo	$O(bd)^{\ddagger}$

[†] Para árvores finitas. * Contra oponente ótimo.

[‡] Análise usual quando sucessores são mantidos em memória; geração sob demanda pode aproximar-se de $O(m)$ ou $O(d)$.
 b : fator de ramificação; m : prof. máxima; d : prof. limite.

Pergunta 17

Na análise usual, o Alfa-Beta tem a mesma complexidade de espaço que o Minimax ($O(bm)$), mas pode ter complexidade de tempo muito menor ($O(b^{m/2})$ no melhor caso). Por que a complexidade de espaço não melhora com a poda nessa análise? O que exatamente o Alfa-Beta mantém em memória que o Minimax também mantém?

12 Atividades Intra-Classe

As atividades abaixo devem ser resolvidas em grupo, com foco em explicar o raciocínio passo a passo, e não apenas em obter a resposta final.

12.1 Aquecimento

Antes das atividades principais, responda rapidamente:

- Qual é o valor de um nó MIN com filhos de utilidades 4, 9 e 2?
- Se $\alpha = 5$ e um nó MIN já encontrou $v = 3$, pode haver poda? Por quê?
- Qual é a diferença essencial entre um estado terminal e uma folha de corte no H-Minimax?

12.2 Cálculo do valor Minimax e poda alfa-beta

Considere a árvore de jogo da Figura 9. Os nós circulares representam estados MAX, os retangulares representam estados MIN e as folhas mostram as utilidades terminais que devem ser propagadas para a raiz.

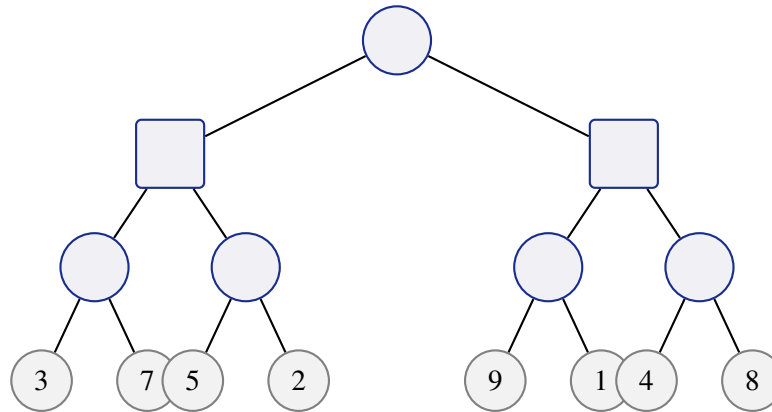


Figura 9: Árvore de jogo da atividade de cálculo Minimax e poda alfa-beta. Estados circulares são de MAX, retangulares são de MIN, e as folhas exibem utilidades terminais.

- Calcule o valor minimax de cada nó interno, de baixo para cima, detalhando cada passo como nos Exemplos 1 e 2.
- Qual é a melhor ação para MAX na raiz? Como você extrai essa ação a partir dos valores calculados? Se MAX e MIN jogarem otimamente, qual será a utilidade final?
- Aplique a poda alfa-beta (esquerda para direita) preenchendo uma tabela como a do Exemplo 2. Para cada poda, indique qual condição ($v \leq \alpha$ ou $v \geq \beta$) a causou e por que é válida.
- Refaça a execução usando a ordenação ideal para a poda alfa-beta: nos nós MAX, examine primeiro os filhos de maior valor minimax; nos nós MIN, examine primeiro os filhos de menor valor minimax. Houve mais poda? Explique.

12.3 Exercício de poda alfa-beta

Considere a árvore de jogo da Figura 10. A raiz é MAX, os três nós internos no nível seguinte são MIN e as folhas exibem as utilidades que devem ser examinadas da esquerda para a direita.

- Execute o alfa-beta passo a passo (esquerda para direita), preenchendo a tabela de rastreamento como no Exemplo 2. Para cada poda, indique qual condição ($v \leq \alpha$ ou $v \geq \beta$) a causou.
- Quais folhas foram avaliadas? Quais foram podadas? Liste os índices (l_1 a l_9).
- Verifique que o resultado é idêntico ao do Minimax completo calculando este último independentemente.

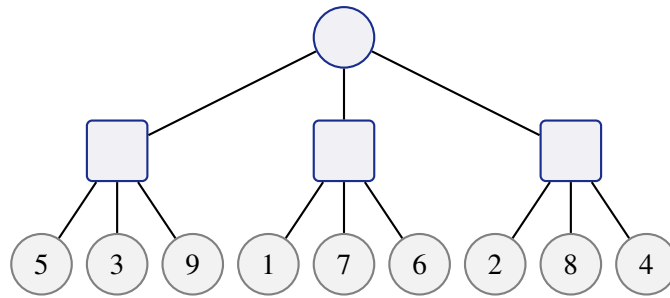


Figura 10: Árvore de jogo para o exercício de poda alfa-beta. A raiz é MAX, os três nós internos no nível seguinte são MIN e as folhas exibem as utilidades usadas na execução da esquerda para a direita.

12.4 Projeto de função de avaliação

Considere o jogo da velha (3×3).

- Proponha uma função de avaliação linear $EVAL(s)$ para esse jogo. Defina explicitamente pelo menos três características f_i e os pesos w_i , justificando cada escolha com base nos três requisitos da Seção 8.
- Sua função satisfaz cada um dos três requisitos? Justifique.
- Calcule $EVAL(s)$ para o estado em que X ocupa (1, 1), (1, 3) e (2, 2), e O ocupa (1, 2) e (3, 2). Apresente os resultados em uma tabela como a do Exemplo 3.
- Descreva um cenário concreto em que o efeito de horizonte (com profundidade $d = 4$) levaria o agente a tomar uma decisão ruim; identifique a posição específica em que a busca parou e o que aconteceria no lance seguinte.

Algoritmo 2 Minimax com Poda Alfa-Beta

```
1: função ALPHA-BETA-SEARCH( $s$ ):
2:    $\alpha \leftarrow -\infty$ 
3:    $\beta \leftarrow +\infty$ 
4:    $best\_action \leftarrow \text{NULL}$ 
5:   for cada  $a \in \text{ACTIONS}(s)$  do
6:      $v \leftarrow \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta)$ 
7:     if  $v > \alpha$  then
8:        $\alpha \leftarrow v$ 
9:        $best\_action \leftarrow a$ 
10:    end if
11:  end for
12:  return  $best\_action$ 

13: função MAX-VALUE( $s, \alpha, \beta$ ):
14:  if  $\text{TERMINAL-TEST}(s)$  then
15:    return  $\text{UTILITY}(s)$ 
16:  end if
17:   $v \leftarrow -\infty$ 
18:  for cada  $a \in \text{ACTIONS}(s)$  do
19:     $v \leftarrow \max(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
20:    if  $v \geq \beta$  then
21:      return  $v$  {poda  $\beta$ }
22:    end if
23:     $\alpha \leftarrow \max(\alpha, v)$ 
24:  end for
25:  return  $v$ 

26: função MIN-VALUE( $s, \alpha, \beta$ ):
27:  if  $\text{TERMINAL-TEST}(s)$  then
28:    return  $\text{UTILITY}(s)$ 
29:  end if
30:   $v \leftarrow +\infty$ 
31:  for cada  $a \in \text{ACTIONS}(s)$  do
32:     $v \leftarrow \min(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$ 
33:    if  $v \leq \alpha$  then
34:      return  $v$  {poda  $\alpha$ }
35:    end if
36:     $\beta \leftarrow \min(\beta, v)$ 
37:  end for
38:  return  $v$ 
```
